SOC L1 LAB · INC-003

Detection, Mitigation, and Triage Guide: INC-003

Catching the request, not just the file that landed

INCIDENT	INC-003, Web shell dropped and executed
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise, sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Detection model
2. Rule-by-rule
3. Tuning
4. Triage playbook
5. Severity matrix
6. Containment and mitigation
7. Escalation template
8. Post-incident
9. Common mistakes
10. Coverage map
11. Metrics

Audience: an L1 analyst who receives the alert, and an L2 or detection engineer who owns the web rules.

This guide uses INC-003 as the worked example. Rule IDs are Wazuh 4.14.7 stock unless marked custom.

---

1. Detection model

A web shell produces four different signals. They answer different questions.

Layer	Source	Question	INC-003 evidence
File	FIM on `/var/www/html`	Did a file appear or change?	100128, 554, 550
Process	auditd execve	What ran, and as whom?	80792 `cp`, `id`, `uname`
Web	nginx access log	Was the shell requested?	100127, 31100/31101
Network	Suricata	Did a known tool or payload appear?	86601 sqlmap

The drop and the use are separate events. An analyst who only looks at FIM will know a file exists but not whether it was used.

1.1 Signal hierarchy

```
Low     31101  a 4xx/5xx web response
        554    a file was added
        ----  inventory; many benign causes

Medium  100128 a .php file appeared in a monitored path
        80790  audit created the file
        ----  candidate; check the deploy ticket

High    100127 .php?cmd= request
        80792  id/uname/curl executed by www-data
        5402   sudo cp into the webroot
        ----  treat as compromise until proven otherwise
```

---

2. Rule-by-rule

2.1 Rule 31100 / 31101: stock web rules

- 31100 is the level-0 grouping rule for access logs. It is the parent for custom rules.
- 31101 alerts on 4xx/5xx responses. A successful web shell request is 200, so it does not fire.
- INC-003 produced 31101 only for the unrelated `/does-not-exist` 404.

2.2 Rule 100127: `.php?cmd=` request

```
<rule id="100127" level="12">
  <if_sid>31100</if_sid>
  <match>.php?cmd=</match>
  <description>Web: command parameter in a PHP request (possible web shell)</description>
  <mitre><id>T1505.003</id><id>T1059.004</id></mitre>
  <group>web,attack,</group>
</rule>
```

- **Parent**
  31100, so the web decoder has already parsed the line.
- **Match**
  A literal `.php?cmd=`.
- **Tested**
  22:12:25 on `GET /updates2.php?cmd=id`. 22:12:28 on the same request with a sqlmap user-agent.
- **False positive**
  `GET /index.php?cmd=status` on a legitimate application fires it. This was confirmed in the §5 test.
- **Gap**
  `?foo=1&cmd=id` does not match. POST bodies are not in the access log.

Why literal: Wazuh uses one `<match>` per rule. Two `<match>` tags do not AND. OS_Regex variants (`\.php.*cmd=`, negated classes) did not match the access log in this version. The literal is the reliable form.

2.3 Rule 100128: PHP file added or modified

```
<rule id="100128" level="12">
  <if_sid>550,554</if_sid>
  <match>.php</match>
  <description>FIM: PHP file added under a monitored path (possible web shell)</description>
  <mitre><id>T1505.003</id></mitre>
  <group>syscheck,web,persistence,</group>
</rule>
```

- **Parents**
  550 (modified) and 554 (added).
- **Match**
  `.php` in the FIM log.
- **Tested**
  22:12:20.956 (`updates2.php` added) and 22:14:11.305 (`test.php` modified).
- **False positive**
  Every legitimate PHP deployment or update. Also any `.php` file under `/home` or `/etc`, because the match is not restricted to the webroot.
- **Gap**
  `.phtml` does not contain `.php` and is missed. `.php5` does contain `.php` and is caught.

2.4 Supporting stock rules

Rule	Use
80790	Audit created the file. Confirms the path
80792	The `cp`, `id`, `uname`, `curl` command trail
5402	Who copied the shell into the webroot
86601	Suricata signature, useful when the tool is known

---

3. Tuning

3.1 Route 100128 to deployment-aware triage

100128 is a deployment monitor first. A release pipeline that writes PHP will always fire it. Do not suppress it. Instead:

- attach the change ticket or deployment ID to the alert.
- send it to a queue that expects release events.
- keep it at level 12 because an unexpected PHP write is high risk.

A rule that is always suppressed is a rule that cannot catch the one write that matters.

3.2 Narrow 100127 for known applications

If an application legitimately uses `cmd` as the first query parameter, allow-list that path. Do not remove the rule. Example:

```
<!-- Example only: verify the path and parameter before deploying - to
<rule id="100129" level="0">
  <if_sid>100127</if_sid>
  <match>/known-app/index.php?cmd=</match>
  <description>Allow-listed application cmd parameter</description>
</rule>
```

3.3 Add variants, not one broad match

Wazuh will not AND multiple `<match>` tags. Add separate rules for:

- `.php?exec=`, `.php?command=`, `.php?shell=`.
- `.phtml?cmd=`.
- POST-body parameters (needs an application log or WAF, not the access log).
- parameter order (`?foo=1&cmd=`), which the literal cannot cover.

3.4 Use the right log source for the question

The nginx access log contains the request line, not the body. A POST-only web shell is invisible here. Add:

- application request logging for the parameters the app accepts.
- a WAF or reverse proxy that inspects the body.
- PHP-FPM access logs if configured.

---

4. Triage playbook

4.1 Five questions

1. What is the file? Path, hash, owner, size, deployment status.
2. Did it run? Audit commands under the web user, and HTTP 200 responses.
3. As whom? `www-data` is not root. Check `uid`, `sudo`, and outbound connections.
4. Is there persistence or a second file? Check the webroot, cron, systemd, and writable directories.
5. Was it deployed? Match the file hash to the pipeline or change ticket.

4.2 Decision tree

```
PHP file added / web request
│
├─ Matches an approved deployment
│    └─ Close with the change ID. Verify the hash
│
└─ Not approved
     ├─ .php?cmd= request or www-data process activity: High. Contain
     ├─ Only the file, no use:  High. Investigate the writer
     ├─ Privilege escalation or second file:  Critical
     └─ Outbound connection from www-data:  Critical. Scope egress
```

4.3 Step by step

1. **Read the FIM event and the raw access line**
2. **Hash the file before deleting it**
3. Check the process trail for the web user (`uid=33`).
4. Check the webroot for other unexpected files.
5. Check persistence before deleting: cron, systemd, `.user.ini`, `.htaccess`, writable upload directories.
6. Assign severity with §5.
7. **Contain**
   Remove the shell, block the source if appropriate, preserve evidence.
8. Escalate with the template in §7.

4.4 What a good ticket contains

- path, SHA-256, owner, and deployment status.

- the exact HTTP request and response code.
- the commands run and the user context.
- whether the file was approved.
- other files in the webroot.
- containment actions and times.

---

5. Severity matrix

Situation	No execution	Execution as web user	Root / persistence
New PHP file, no request	Medium	High	Critical
`.php?cmd=` request, 200	High	High	Critical
Web shell plus outbound connection	High	Critical	Critical
Legitimate deploy, hash matches	Low	,	,

INC-003: new file, execution as `www-data`, no root, no persistence to High.

Raise to Critical if the web user can `sudo`, if the host holds credentials, or if a second payload or persistence is found.

---

6. Containment and mitigation

6.1 During the incident

Action	Command	Caution
Hash the shell	`sha256sum /var/www/html/updates2.php`	Before deletion
Preserve the request	copy the nginx line and the alert JSON	Before rotation
Remove the shell	`rm /var/www/html/updates2.php`	After hashing
Check for siblings	`find /var/www/html -name '*.php' -newermt ...`	Look for uploads
Check the writer	audit `cp`, `tee`, `mv` around the file time	Who put it there
Check persistence	cron, systemd, `.user.ini`, `.htaccess`	Before deleting the home

6.2 Host and application hardening

```
# disable dangerous functions (php.ini; restart FPM)
# disable_functions = system,exec,shell_exec,passthru,popen,proc_open
# open_basedir = /var/www/html:/tmp
```

```
# make the webroot read-only at runtime; deploy as a separate user
# (implementation depends on the deployment tool)
```

- Separate the build/deploy user from the runtime user.
- Do not let the web user write executable files.
- Disable PHP execution in upload directories.
- Put a WAF or reverse proxy in front of the application.
- Restrict outbound traffic from the web tier.

6.3 Detection controls

Control	Purpose
100128	New or modified PHP file
100127	`.php?cmd=` request
80790/80792	File creation and command trail
5402	Who wrote the file with sudo
86601	Known tool signatures
Key/deploy hash baseline	Distinguish a release from a shell

---

7. Escalation template

```
INC-003: Web shell dropped and executed

What:  /var/www/html/updates2.php (SHA-256 50e626...) was copied into the webroot at
       22:12:20 UTC and executed as www-data. GET /updates2.php?cmd=id returned
       uid=33(www-data) at 22:12:24.
Scope: Host soc-victim, webroot /var/www/html, web user www-data.
Detect: 100128 at 22:12:20, 100127 at 22:12:25/28, 80790 created file, 80792 cp/id/uname.
Done:  File hashed and removed, webroot checked, evidence preserved.
Ask:   (1) Was any PHP deploy approved at that time? (2) Check other hosts for the same
       hash. (3) Review webroot write permissions and disable_functions.
```

---

8. Post-incident

- Add the shell hash to a blocklist or file-hash watch.
- Add `.phtml` and parameter-order variants to the rule tests.
- Confirm the webroot is clean and the deployment user is separated.
- Review whether the app needs PHP execution at all.
- Measure: file write to 100128, request to 100127.
- Update the runbook with the deployment-ticket check.

9. Common mistakes

Mistake	Consequence
Deleting the shell before hashing	Evidence lost
Treating 100128 as an attack without checking the deploy ticket	False positives erode trust
Closing on "it is only www-data"	Web user can read configs and reach internal services
Relying on Suricata alone	A plain curl request has no signature
Relying on the access log for POST shells	The body is not logged
Adding two `<match>` tags	Only one is evaluated
Suppressing 100128 on a busy deploy day	You may suppress the one real shell

10. Coverage map

Technique	ID	Covered by	Gap
Web shell	T1505.003	100127, 100128	`.phtml`, POST, parameter order
Unix shell	T1059.004	80792	Noise. Needs session filtering
Discovery	T1082, T1033	80792 `uname`, `id`	No sequence rule
Ingress tool transfer	T1105	not covered	Add egress/proxy logs
Valid accounts	T1078	5715, 5402	Admin vs attacker by IP

11. Metrics

Metric	INC-003
File added to 100128	<1 s
Request to 100127	<1 s
Clean-window alerts	67
Broad session alerts	~3,900 (PHP-FPM install noise)
Execution context	`www-data`
Time to containment	minutes (exercise)

The detection worked once the web rule existed. The larger lesson is that the first run had no web-layer signal at all: the file and the process were visible, the successful request was not.